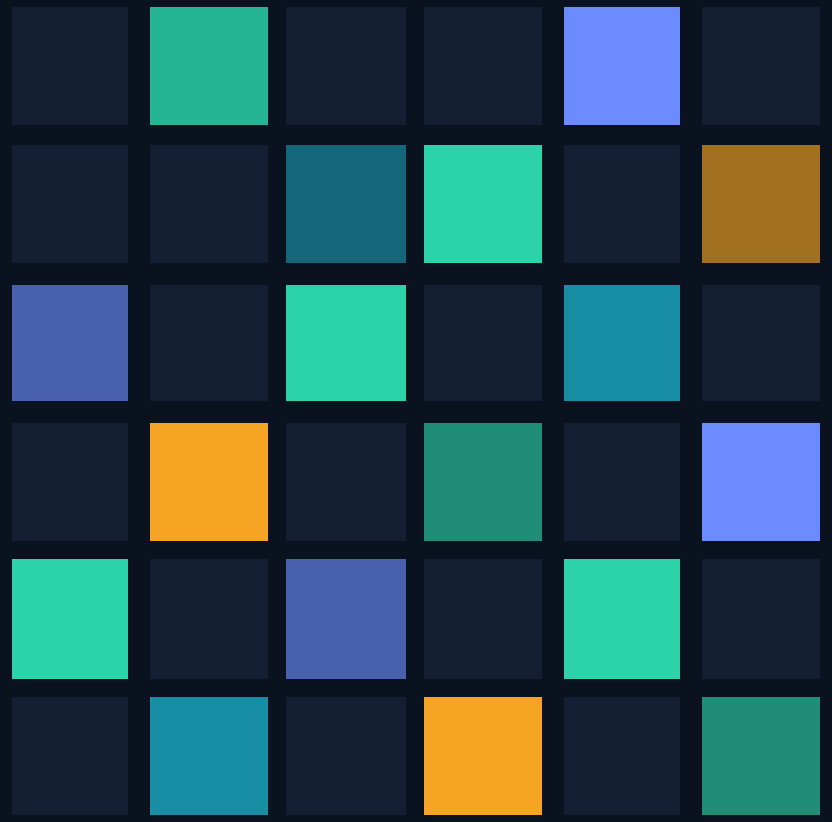
STP · AI ADOPTION IN SPACE SYSTEMS

AI-Assisted Engineering Design, Software & Hardware Development



Gulshan Gupta, SAC

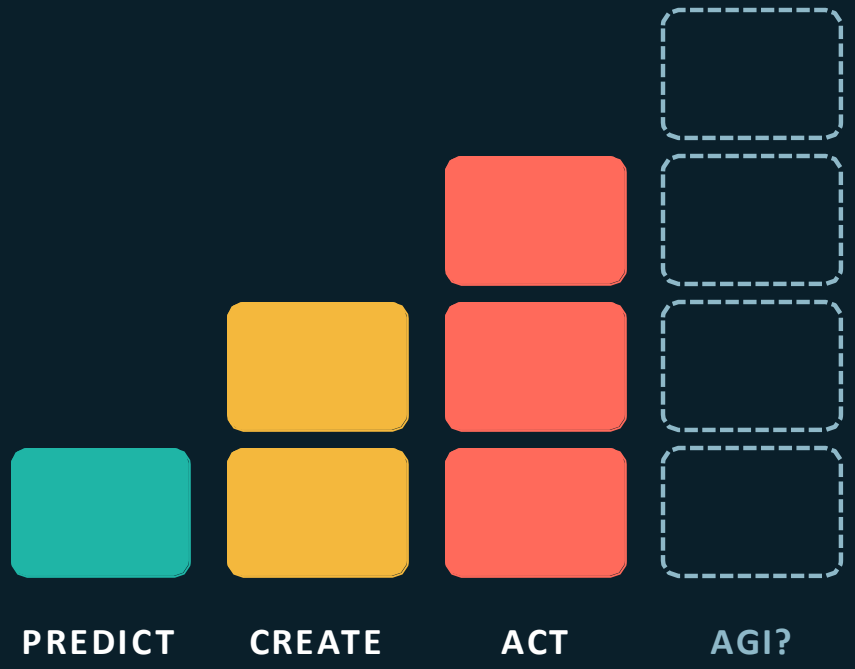
13 October 2026

Outline

01	**Evolution of AI**	• Major Breakthroughs • Predictive vs Generative vs Agentic AI • The trend
02	**AI in Engineering and Software Development**	• Applications of Agentic and Generative AI • AI in Engineering Process and SDLC • Case Studies
03	**Decoding Agentic AI**	• Components of Agentic AI • AI Harness
04	**Secure On-premise AI**	• Running models locally • Securing the Different Layers • Attack Paradigm • Best Practices
05	**On-going Developments at SAC**	• Intelligent Information Retrieval System • Transformer backed Adaptive WAF

Evolution of AI

The shift from analyzing data to executing goals.



THE JOURNEY SO FAR

70 years. Major Breakthroughs



1956 — The name is born

Dartmouth summer workshop coins “artificial intelligence” [1]

2012 — Deep learning wins

AlexNet hits 15.3% ImageNet top-5 error vs 26.2% for the runner-up [2]

2017 — Attention arrives

Transformer paper: 28.4 BLEU, trained in 3.5 days on 8 GPUs [3]

2022 — AI goes mainstream

ChatGPT reaches ~100M monthly users in about 2 months (UBS est.) [4]

2028 forecast — Agents everywhere?

Gartner forecast: 33% of enterprise apps embed agentic AI, up from <1% in 2024 [5]

Sources: [1] Dartmouth Alumni Magazine; NIST [2] Krizhevsky et al., 2012 [3] Vaswani et al., 2017 [4] UBS via Reuters, 2023 [5] Gartner

Predictive AI

Core Function

Forecasts future outcomes.

Mechanism

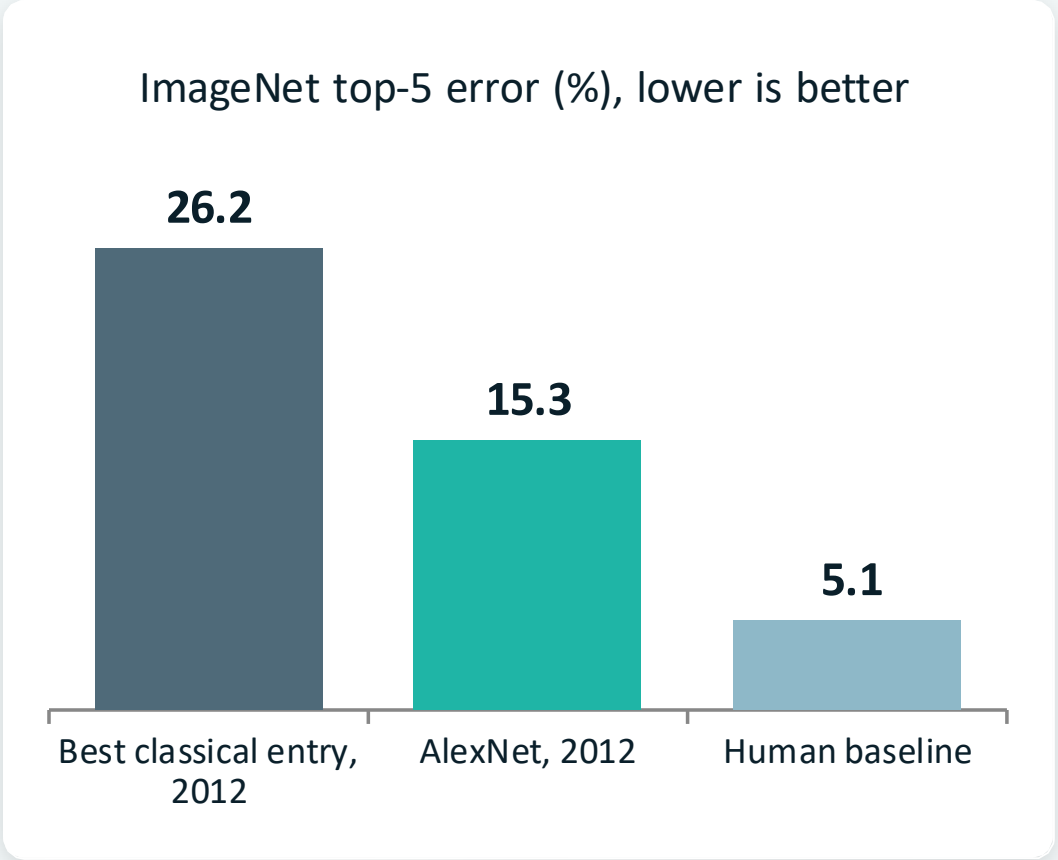
Analyzes historical data.

Output Type

Probabilities and classifications.

Autonomy Level

Zero (requires human action).



Sources: [2] Krizhevsky et al., 2012 (top-5 error 15.3% vs 26.2%) [7] Russakovsky et al., 2015 (human baseline about 5.1%)

THE CREATOR

Generative AI

AUTONOMY



LOW

Core Function

Creates original content.

Mechanism

Synthesizes massive datasets.

Output Type

Text, code, and images.

Autonomy Level

Low (waits for prompts).

MODEL SCALE, 2017 TO 2020



65 M

parameters in the original Transformer (base) [3]



175 B

parameters in GPT-3 [8]

≈ 2,700× in three years

Block sizes are illustrative, not to scale.

Sources: [3] Vaswani et al., 2017 (base model about 65M parameters) [8] Brown et al., 2020 (GPT-3, 175B parameters)

THE EXECUTOR

Agentic AI

AUTONOMY



HIGH

Core Function

Acts autonomously on goals.

Mechanism

Orchestrates multi-step workflows.

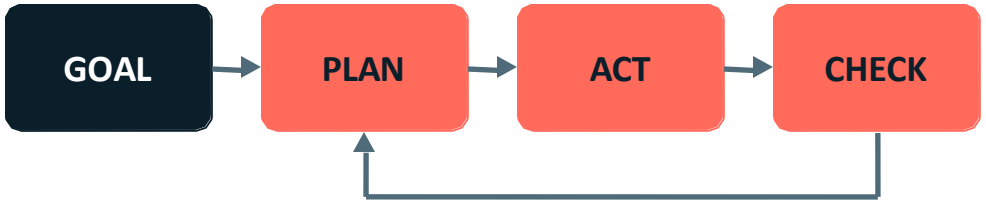
Output Type

API calls and actions.

Autonomy Level

High (goal-driven execution).

THE AGENT LOOP (SIMPLIFIED)



repeat until the goal is met

33%

from <1% in 2024

of enterprise software apps include agentic AI by 2028 (forecast)

15%

from 0% in 2024

of day-to-day work decisions made autonomously by 2028 (forecast)

Sources: [5] Gartner forecasts (AI Agents: Powering New Business Models). The loop diagram is a simplified illustration.

THE PATH FORWARD

Increasing Autonomy

Autonomy: zero

Predictive AI

Pattern recognition and statistical forecasting.

Autonomy: low

Generative AI

Creation of novel text, image and code assets.

Autonomy: high

Agentic AI

Autonomous planning and multi-step execution.

Yet to Achieve

AGI

Human-level generalization across domains.

AT A GLANCE

Comparison AIs

Feature	**Predictive AI** The Analyst	**Generative AI** The Creator	**Agentic AI** The Executor
Output	Probabilities and labels	Text, images and code	API calls and actions
Input trigger	Historical data feed	Human prompt	Broad goal assignment
Autonomy	Zero	Low	High
Use case	Churn forecasting	Drafting emails	Autonomous Code updates
Engineering example	Forecasting equipment faults from telemetry	Drafting test cases and design documents	Keeping requirements traceability up to date

THE TAKEAWAY

Spot. Create. Execute



Predictive spots the problem.



Generative creates the solution.



Agentic gets it done.

AI in Engineering and Software Development

Generative and Agentic AI for design, software and hardware in space systems



ROADMAP

Apply it, embed it, learn from cases

PART 1

Applications of generative and agentic AI

- Generative versus agentic AI
- Design, software and hardware
- Space-system lifecycle
- Optimisation, traceability, documentation
- Practical tools

PART 2

AI in the software development life cycle

- The V-model with AI at each leg
- SDLC usage table
- Code and test generation
- Agentic test loop in CI
- Threats and guardrails

PART 3

Case studies

- Airbus and GM generative design
- NVIDIA ChipNeMo
- RAG for space requirements
- Measured productivity evidence

PART 1

Applications of generative and agentic AI

What each kind of AI does across engineering design, software and hardware



FOUNDATIONS

Generative AI produces; agentic AI acts

GENERATIVE AI

Produces artifacts on request

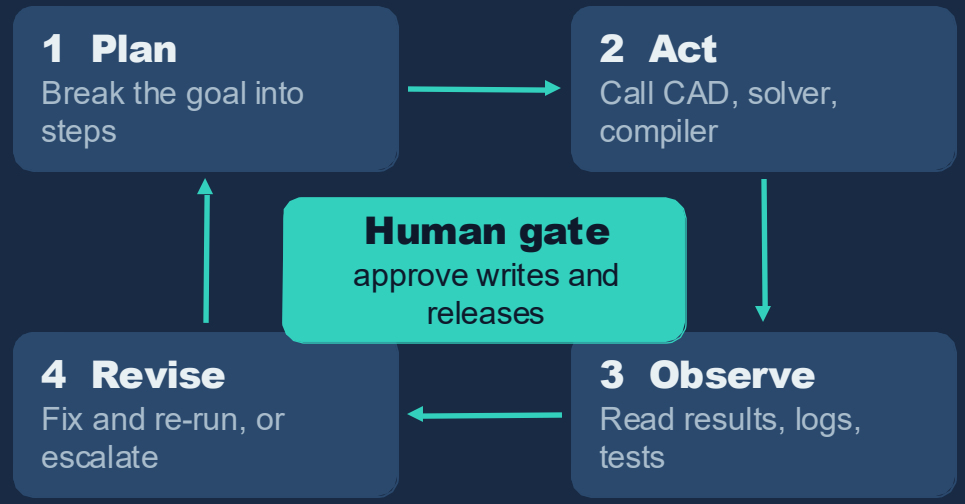
- Code, test drafts, CAD variants, documents
- One prompt, one response; a person drives each step

AGENTIC AI

Pursues a goal with tools

- Plans, calls tools, checks results, iterates
- Humans approve at defined gates

THE AGENT LOOP



THREE ENGINEERING DOMAINS

Where it applies: design, software, hardware

Engineering design

- Concept variants from requirements and constraints
- Parametric CAD and generative topology exploration
- Trade-study drafting with LLM-assisted literature search
- Design-review checklists auto-populated

Software development

- Code generation and refactoring in the IDE
- Unit, integration and regression test drafting
- Agents that open PRs, run CI and fix failures
- Legacy code explanation and migration

Hardware development

- RTL, HDL and testbench generation
- EDA script and constraint authoring
- Datasheet Q&A and BOM / part selection support
- Log triage for lab bring-up and HIL test

Generative AI produces artefacts (code, variants, documents). Agentic AI plans, calls tools, checks results and iterates, with humans approving at defined gates.

AI across the space-system lifecycle

Concept and design

- Mission and architecture trade studies at speed
- Generative structures for mass-critical brackets and trusses
- Requirement-to-design consistency checks

Analysis

- ML surrogates for CFD, FEA and thermal models
- Faster orbit, link-budget and Monte Carlo sweeps
- Uncertainty-aware models to bound risk

Build and test (AIT)

- AIT and test-procedure generation from specs
- Configuration and non-conformance report triage

Test and flight data

- Anomaly detection on test and telemetry data
- Evidence packages for review boards

Human sign-off stays mandatory · AI-generated evidence traceable and reproducible

PRACTICAL APPLICATIONS

Tools for better productivity and decisions

ACTIVITY	TOOL CATEGORY (EXAMPLES)	DECISION VALUE
Coding and refactoring	IDE assistants and agents (GitHub Copilot, Cursor, Claude Code, Continue)	Faster iteration; more time on architecture
Testing and QA	Test generators, mutation and fuzz tools with LLM front-ends	Earlier defect discovery; coverage visibility
Simulation and design	Surrogate models, optimisation suites, generative design in CAD	Wider design space explored per schedule
Knowledge and documents	Enterprise RAG search, document copilots	Faster answers from past projects and standards
Data and telemetry	ML anomaly detection, notebooks with AI assist	Evidence-based go / no-go decisions

PART 2

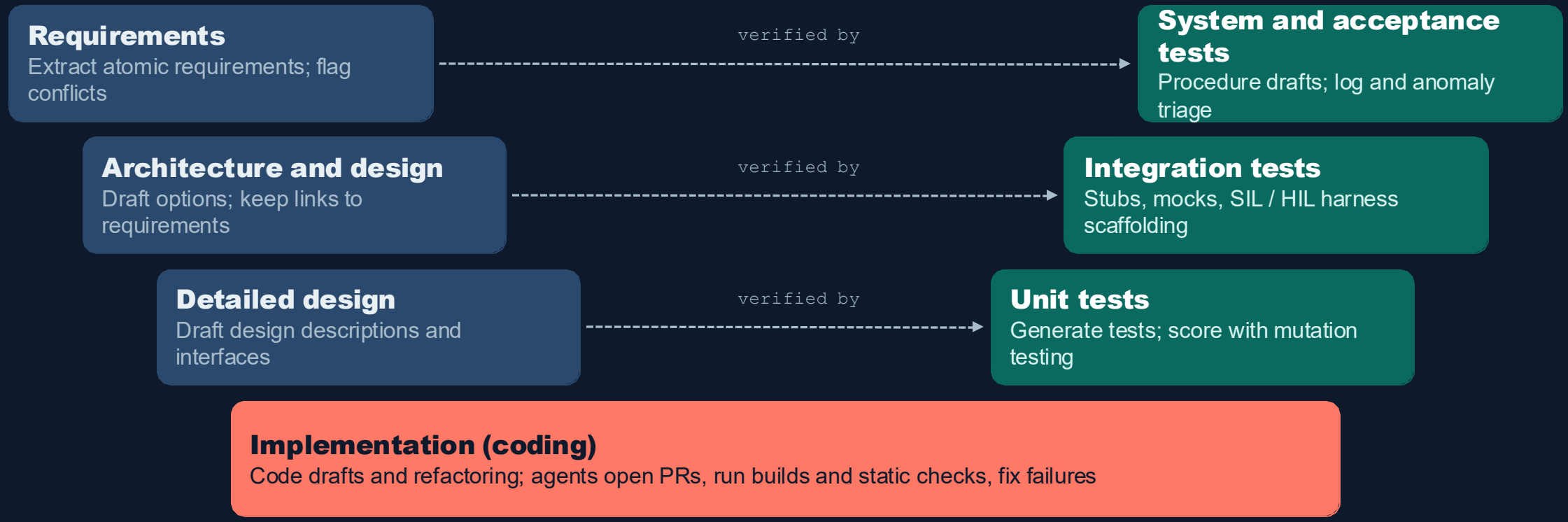
AI in the software development life cycle

Where generative and agentic AI fit, and where people keep the gates



Assist every phase

AI assists at every step



Reviews at each gate (for example SRR, PDR, CDR, QR, AR in ECSS-E-ST-40C): the AI drafts, an engineer approves.

Generative and agentic AI across the SDLC

SDLC PHASE	GENERATIVE AI · PRODUCES	AGENTIC AI · ACTS WITH TOOLS	HUMAN GATE AND ASSURANCE
Requirements	Extract atomic requirements from specs and ICDs; rewrite ambiguous text	Check for conflicting, orphaned and untested requirements; run change-impact analysis	Requirements review; baseline approval
Architecture and design	Draft design options, trade-study text and interface descriptions	Run parametric sweeps and surrogate optimisation; set up and summarise simulations	Design review; cross-check against solver
Implementation	Code drafts, refactoring, legacy-code explanation and migration	Open pull requests; run builds and static analysis; fix failures	Code review; MISRA / project rules in CI
Verification and testing	Unit, boundary and fault-case tests from requirements; stubs and mocks	Loop: run tests, read coverage and mutation score, refine to target	Reviewed tests; mutation score, not only coverage
Integration and release	Release notes, change summaries, SIL / HIL harness scaffolding	Triage CI failures; propose fixes; assemble release candidates	Configuration control; release-gate sign-off
Documentation and traceability	Design descriptions, test reports and manuals from source artefacts	Keep documents and trace links in sync as code and requirements change	Engineer approves links; audit trail
Operations and maintenance	Explain logs; draft anomaly and non-conformance reports	Triage test and telemetry anomalies; gather evidence	Anomaly board decision; reproducible evidence

GOVERNING AGENTS

Threats and guardrails for coding agents

Poisoned or tampered models

Back door weights; pickle files can run code on load; typo squatted repos

Prompt injection

Malicious text in comments, docs, issues; agents may follow embedded instructions

Data leakage

Telemetry and logs hold IP; shared caches and adapters across teams

Insecure serving

Local servers often lack authentication; binding to all interfaces exposes them

Unsafe agent actions

Shell, file, install tools; auto-approved commands; ex-filtration via tool calls

Insecure generated code

Vulnerable patterns, hallucinated packages, echoed secrets, license issues

GUARDRAILS

Allowlist and sandbox
Commands, paths, packages; step and time limits

Human approval
Writes, shell, network; review every diff

Untrusted input
Repo and docs are data; never auto-run retrieved commands

Data and audit
Secret scanning; export-control rules; log prompts and tool calls

PART 3

Case studies

Documented adoption in structures, chip design, requirements and software productivity



CASE STUDIES 1 AND 2 · GENERATIVE DESIGN

Generative design cuts mass in real parts

AIRBUS · A320 BIONIC PARTITION

~45% **lighter (about 30 kg) than the current design**

- Generative design produced thousands of alternatives in the cloud
- 3D printed in Scalmalloy, an aluminium-magnesium-scandium alloy
- Airbus and Autodesk, 2016

GENERAL MOTORS · SEAT BRACKET

~40% **lighter and ~20% stronger; 8 parts become 1**

- More than 150 alternative designs generated
- One stainless-steel part made by additive manufacturing
- Autodesk Fusion 360 generative design

Pattern: engineers set constraints and goals; AI widens the option space; engineers select, verify and sign off. Figures are as reported by the companies.

Source: [10] Airbus (2016). Breakthrough for future Airbus A320. airbus.com/en/newsroom/news/2016-03-pioneering-bionic-3d-printing
[11] Autodesk. General Motors generative design customer story. autodesk.com/customer-stories/general-motors-generative-design

CASE STUDY 3 · LLMs FOR ENGINEERING

NVIDIA ChipNeMo: LLM adapted to chip design

ADAPTATION PIPELINE

Base LLaMA2 models
General-purpose starting point

→

Domain tokenizer and continued pretraining
On internal design data

→

Alignment
Supervised fine-tuning on domain instructions

→

Domain-adapted retrieval
RAG over internal documents

THREE APPLICATIONS

Engineering assistant chat
Answers design questions

EDA script generation
Tool scripts from natural language

Bug summarisation and analysis
Condenses and triages bug reports

Reported result: ChipNeMo-70B beats GPT-4 on the assistant chatbot and EDA script generation; competitive on bug summarisation.

Source: [13] Liu et al., ChipNeMo: Domain-Adapted LLMs for Chip Design, arXiv:2311.00176.

CASE STUDY 4 · SPACE REQUIREMENTS

RAG for requirements from space standards

Mission document
Space rideshare payload user guide

→

Classify
Paragraphs into categories: payload, orbit, on-board data

→

Filter standards
50+ ECSS documents narrowed to 17

→

Retrieve
Category-extended queries pull relevant text

→

Generate
LLM drafts requirements

REPORTED OUTCOME

50+ → 17

ECSS documents kept by similarity filtering; expert review found no relevant document missed, though some marginal ones were kept

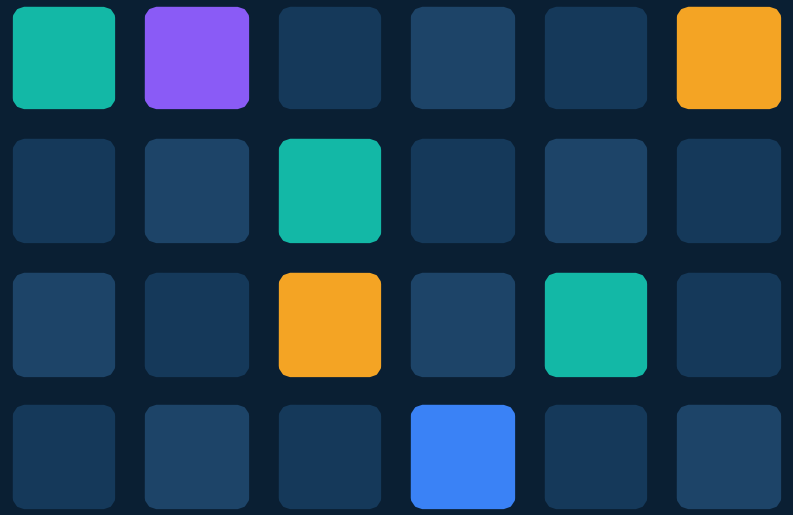
CAUTION

- One case study; qualitative expert review; no ground truth
- No precision, recall or effort-reduction figures reported
- Traceability and auditability of generated requirements are future work

Source: [14] Arora et al. (Monash University; Starbound Space Solutions), arXiv:2507.07689.

TECHNICAL SEMINAR · OCTOBER 2026

Agentic AI and the Execution Harness



Agentic components · tools and telemetry · execution harness

Agentic AI Architecture

Integrating reasoning, memory and secure execution harnesses



Cognitive & planning core

SLMs and LLMs drive intent recognition and multi-step task decomposition



Memory & state persistence

High-performance multi-tenant vector databases provide long-term retrieval and execution continuity across complex agent sessions.



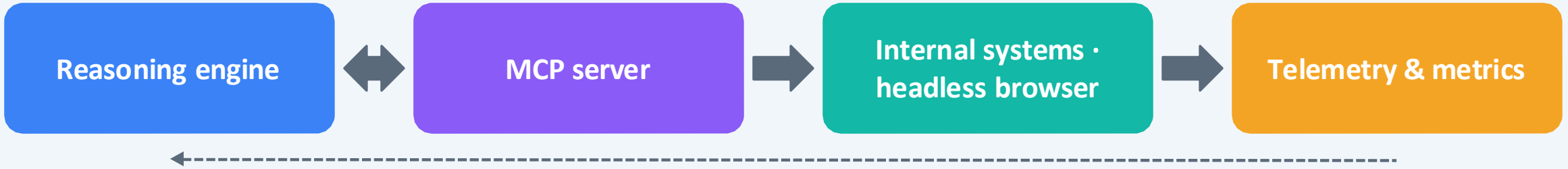
Execution harness

Deterministic, isolated environments where agent logic evaluates real-time system state and computes the next sequence of operations.

Critical-operations boundary: reasoning, memory and execution stay inside one secure enclave.

Secure tool integration and telemetry loops

Enabling autonomous action and self-correction



Feedback loop: the plan adapts on task failure or unexpected state changes

Model Context Protocol

Standardized, bi-directional communication between the reasoning engine and internal systems, APIs or databases.

Autonomous action

Headless frameworks (Playwright, Chrome DevTools Protocol) orchestrate multi-step system interactions without human intervention.

Continuous feedback

Telemetry and execution metrics feed the perception layer, so planning adapts dynamically to failures and state changes.

THE HARNESS DEFINED

Hardened boundary around a stochastic model

Actuation, observation and isolation for an LLM's non-deterministic reasoning core, bridging semantic space with physical computing environments.

System = $\pi\theta$ Policy + τ Cognitive scaffolding + $\mathcal{E}$ Harness / runtime

Agentic equation

Ephemeral sandboxing

Kernel or hypervisor isolation (gVisor, microVMs) bounds code, filesystem and syscalls.

Protocol mediation

Strongly-typed interfaces such as MCP for tool injection, schema validation and serialization.

Trajectory ledger

Event records support the generation & management of offline RLHF/DPO datasets.

Deterministic governance

OS-level token, compute and memory quotas, network egress control and HITL interrupts.

Secure On-premise AI

Local LLM deployment, air-gapped operations and inference security



GPU memory blocks: the scarce resource

Run it privately. Defend it

PART 1

Running LLMs locally

- Memory and hardware sizing
- Model selection by engineering task
- Quantization
- Latency and developer experience
- Zero-egress architecture
- Updating an air-gapped environment

PART 2

Securing inference

- The recursive resource-exhaustion trap
- Defenses before the token limit
- Hardening the runtime: containers, network, model files
- LLM DoS versus traditional IT DoS
- Asymmetry and statefulness

HARDWARE SIZING

LLMs are memory-bound, not compute-bound

INPUT

1 billion parameters

stored at FP16

≈

WEIGHTS

~2 GB

of VRAM

+

CONTEXT

KV cache

grows with the context window

VRAM math :
memory capacity decides what fitsmemory bandwidth decides how fast it runs.

PRACTICAL EXAMPLE

Sizing a 7B model in pure FP16

24 GB of VRAM on an RTX 4090, with the KV cache drawn at its 4 GB upper bound



MODEL WEIGHTS

~14 GB

in FP16, before any prompt

KV CACHE

+2–4 GB

for a standard context window

TOTAL

≈16–18 GB

within a 24 GB RTX 4090

CPU VS GPU VS NPU

Three ways to host a model locally

DATA CENTER / WORKSTATION

GPU

- RTX 4090 (24 GB) or A100 (80 GB)
- 1,008 GB/s memory bandwidth
- Over 40 tokens/sec

SYSTEM RAM

CPU

- DDR5 delivers only ~64 GB/s
- 5–15 tokens/sec
- Usable for chat, terrible for autocomplete

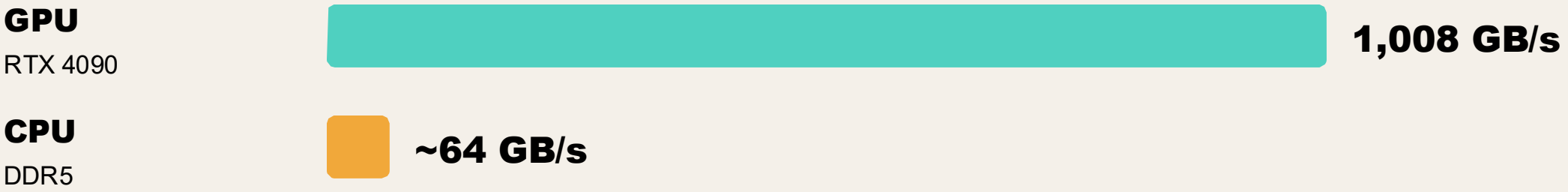
UNIFIED MEMORY

NPU

- M3 Max with 128 GB
- Runs 70B+ models on a laptop
- No server-grade GPU needed

WHY BANDWIDTH MATTERS

Memory bandwidth limits token speed



>40 tok/s

GPU generation, fast enough for autocomplete

5–15 tok/s

CPU output, workable for chat only

MODEL SELECTION

Match model size to the engineering task

LOW LATENCY · HIGH SPEED

Autocomplete

CodeGemma 7B
Falcon H1R 7B

Fast enough for real-time keystroke prediction

~6–8 GB VRAM quantized: a standard laptop

HIGH COMPLEXITY

Agentic reasoning

Llama 3.1 70B
Mistral Large

Multi-step reasoning and root-cause analysis

Internal server: 2x to 4x RTX 4090 or A6000

BALANCED WORKLOAD

General and docs

GPT-OSS 20B
Phi-3 3.8B

Release notes and inline docstrings

Very low hardware overhead

QUANTIZATION

Making models fit: from 16-bit to 4-bit



Llama 3 8B: ~16 GB becomes ~4.5 GB, trading a little smartness for budget.

THE TRADE-OFF

Cost of Compression

~98%

8-bit quantization

retains the model's logic

~90%

4-bit quantization

perfect for code boilerplate

LATENCY AND UX

Latency is the biggest UX challenge

TIME-TO-FIRST-TOKEN

< 250 ms

Target for code assistance

GENERATION THROUGHPUT

> 40 tok/s

Minimum sustained speed

LATENCY REDUCTION

Three techniques that cut latency

FlashAttention-2

Optimizes GPU memory reads for long-context tasks like log analysis

Faster log analysis

PagedAttention + continuous batching

Standard in vLLM: cuts KV cache memory waste by up to 50%

10–20 devs, one GPU

Context capping

For inline autocomplete, send only surrounding code, not the full 32k window

2,000 tokens max

FlashAttention-2 is roughly 2x faster than the original FlashAttention.
The vLLM paper reports near-zero KV-cache waste and 2–4x higher throughput than earlier serving systems.

LOCAL AI SERVER SETUP

Server selection

PERSONAL / LOCAL SYSTEM

Ollama or LM Studio

Extremely fast setup.

Loopback Address

CENTRALIZED SERVER

vLLM

On an internal High-end GPU / GPU cluster.

Network Address **OpenAI-compatible API**

Usually hosted behind a firewall or reverse proxy. Authentication to be handled separately

CONNECTING THE IDE

Connect the IDE without external APIs

1

Install

Continue for VS Code

2

Configure

Point the extension at your local or internal endpoint

3

Send

Open files and terminal errors go only to your server

Result: Air-gapped contained Inferencing with zero leakage risk

AIR-GAPPED DOWNLOAD / UPDATES

Moving models across the air gap



Every user can pull and reuse these verified model weights from this internal repository, hosted on network.

Agentic AI in Air-gapped Environment

AIR-GAPPED ZONE

Inference plane

Quantized, self-hosted models on vLLM or TensorRT-LLM.

Execution plane

Open-source harness with stateless sandboxes set to network_mode: none.

Integration plane

Local MCP servers to private VCS (GitLab/Bitbucket), internal docs and vector stores.

Risks Handled

Ex-filtration prevention: Air-gapped isolation guarantees zero leakage of IP, telemetry or PII

Privilege Escalation: Sand-boxing and controlled execution plane prevents privilege escalation

Supply-Chain Compromise: Air-gapped systems can disable unverified remote updates/downloads

Agent Hijacking via External Inputs: Prevents adversarial API responses or remote content from manipulating agent behavior.

Unauthorized Tool Invocation: Air-gapping blocks agents from calling risky online tools or APIs

SECURITY AND INTEGRITY

Treat model files as untrusted until verified

Official weights only

Never trust random repositories

SHA256 both sides

Check checksums before and after the transfer across the air gap.

Mind the pickle

Pickle files can run hidden code when loaded; .safetensors stores pure data.

PART 2

Securing the inference layer

How a handful of requests can exhaust a GPU server, and how to stop them

ATTACK 1 · RESOURCE EXHAUSTION

The Recursive Trap

THE MALICIOUS PROMPT

"Write a step-by-step breakdown of every prime number up to 1,000,000. Do not stop until finished."

KV cache bloat
Endless GPU VRAM consumption

→

Batching hijack
Legitimate queue slots locked

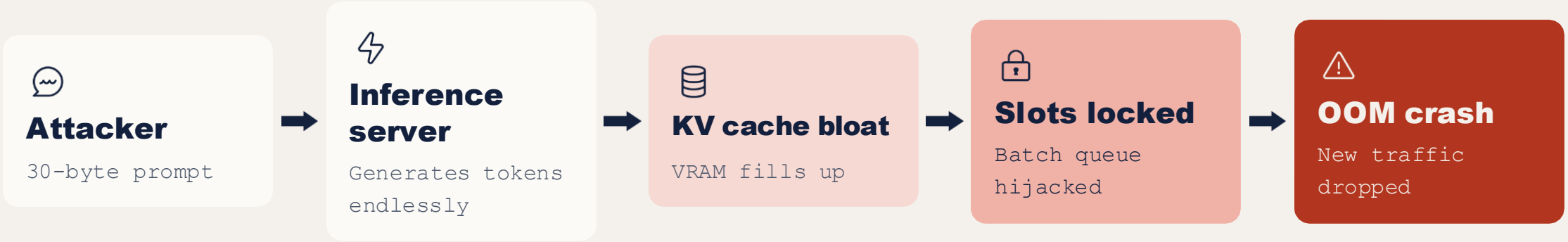
→

OOM failure
System drops new traffic

This maps to OWASP LLM10:2025, Unbounded Consumption: user-controlled requests with no server-side bounds on resource use.

ATTACK WALKTHROUGH

From one short prompt to a dropped service



> Single-digit request volume is enough, and every payload is semantically valid, so **a traditional WAF sees nothing wrong**.

DEFENSE

Preempt the token limit

Repetition detection

Abort on artificial N-gram loops

Frequency penalties

Force mathematical token variation

Concurrency caps

Limit active concurrent sequence counts

Stop the inference process before it exhausts VRAM.

OWASP's guidance for unbounded consumption is to validate and bound user-controlled parameters, such as max_tokens, and input size on the server side.

DEFENSE IN DEPTH

Harden the runtime around the model

Network edge	Bind to loopback or a private IP, allow-list the internal subnet in the firewall, and front the API with an authenticating proxy.
Container sandbox	Run as a non-root user, drop all Linux capabilities, mount the root filesystem read-only and never run privileged.
VRAM allocation	Expose GPUs through the NVIDIA Container Toolkit and give each container only the devices, memory and CPU it needs.
Inference limits	Cap max_tokens, input size and concurrent sequences on the server so no single request can hold the GPU.
Model files	Serve verified .safetensors from the internal mirror, mounted read-only, with no remote-code execution.

THREAT COMPARISON

LLM DoS versus traditional IT DoS

	Traditional IT DoS (L3/L4/L7)	LLM inference DoS
Target	Network edge (bandwidth), load balancers, CPU	GPU VRAM (KV cache) and Tensor Cores
Vector	SYN floods, UDP amplification, HTTP GET floods	Recursive prompts, context stuffing, repetition loops
Volume	Massive volume needed (botnets)	Minimal volume: single digits
Detection	Packet inspection and rate limiting	Evades WAFs: payloads are semantically valid

Unlike web endpoints with predictable costs, LLM request cost varies with input length, output length and requested completions, which is why OWASP lists unbounded consumption as a distinct risk.

ARCHITECTURAL VULNERABILITY

The asymmetry of compute

ATTACKER SENDS

30 bytes

Almost zero compute spent



SERVER MUST SPEND

Teraflops

and gigabytes of memory

In a traditional L7 HTTP flood, attacker and server costs are roughly symmetrical. In an LLM DoS, **the gap is extreme**.

ARCHITECTURAL VULNERABILITY

Statefulness exploitation

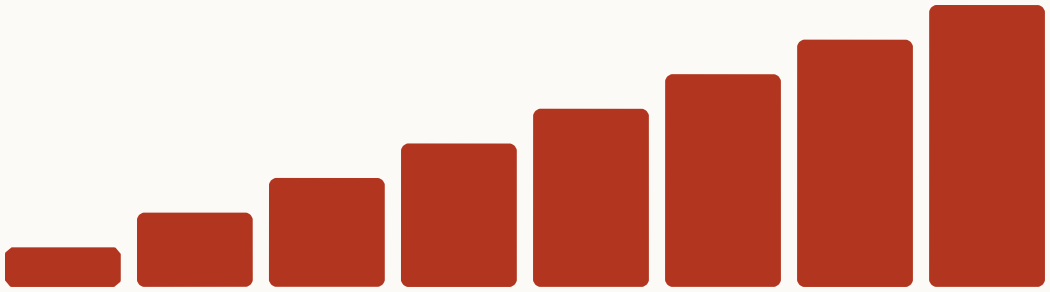
TRADITIONAL L4

Exploits the statefulness of the TCP handshake

LLM DoS

Exploits the statefulness of the neural network itself

KV CACHE IN GPU MEMORY



token 1 token n

Every generated token's attention state stays in VRAM until an End of Sequence (EOS) token arrives.

SUMMARY

Key takeaways

01	**Size by memory**	~2 GB of VRAM per billion FP16 parameters, plus KV cache
02	**Right-size and compress**	Match the model to the task, then quantize to fit the budget
03	**Keep it private**	Zero-egress serving and a verified air-gap pipeline
04	**Defend inference**	Detect loops, penalize repetition, cap concurrency
05	**Harden the runtime**	Containerize, run non-root and read-only, bind privately, bound requests

On-going Development at SAC



Tranformer backed Adaptive Web Application Firewall (WAF)

Objective Replace brittle signature rules (e.g. OWASP CRS) with a transformer WAF that is accurate at low false-positive rates, knows when it is uncertain, and adapts per tenant without sharing raw traffic.

01
Canonical request
Section-marked serialization, 5-pass URL decoding, typed route and value masking

→

02
Extended tokenizer
30,522 BERT tokens + 2,543 HTTP/attack tokens = 33,065

→

03
Frozen encoder
BERT adapted with MLM + LoRA (r=8) merged, then frozen

→

04
Three heads
MLP classifier, CLOSR hypersphere clustering, and a 32-d skip bottleneck

→

05
Meta-learner
Confidence-gated fusion of a 168-d signal vector into calibrated risk + uncertainty

→

06
Decision policy
Based on Uncertainty and Risk: BLOCK, PASS or FLAG

Sharing the encoder

Encoder along with LORA adapters in freezed and can be shared along multiple web applications. This model has been trained with HTTP traffic and is capable to distinguish standard cases

Fusion to handle variety

A naive meta-learner just copies the classifier. A 32-d skip dropped with p=0.5 plus probability-scale gating keeps it a true combiner.

Multi-tenant design

Each tenant trains only classifier heads and meta learner on its own traffic; replay and EWC limit forgetting.

Contextual search over mailbox data, in three pipelines

GOAL
on current mailbox data

Contextual search | **Comprehension** | **Natural language query** | **Correlation**

CHALLENGES TO DESIGN FOR

- Huge data volume
- Dynamic data
- Variety of data (scanned, tables, equations)
- Indic language support
- Strict data security / access controls
- No data leakage between users
- Precision with completeness (recall)
- Original-email references (explainability)
- Multiple parallel users (resource optimization)

CORE DESIGN · MAJOR TASKS

1 Input pipeline

Cleansing

Content extraction
variety of formats

Storage
dynamic updates

2 Retrieval pipeline

Task understanding
natural language processing

Retrieval
fusion, re-ranking

Context augmentation

Domain adaptation
decoding ISRO-related semantics

3 Inferencing pipeline

Provide refined context

Add references
original emails

SOURCES

References-1

1 J. Hertog, “The Birth of Artificial Intelligence,” Dartmouth Alumni Magazine; R. D. Sriram (2006), “Artificial Intelligence in Engineering: Personal Reflections,” NIST.

2 A. Krizhevsky, I. Sutskever, G. Hinton (2012), “ImageNet Classification with Deep Convolutional Neural Networks,” NIPS 2012.

3 A. Vaswani et al. (2017), “Attention Is All You Need,” arXiv:1706.03762.

4 Reuters (1 Feb 2023), “ChatGPT sets record for fastest-growing user base: analyst note” (UBS; Similarweb; Sensor Tower). Spotify figure: Insider report of the same UBS note.

5 Gartner, “AI Agents: Powering New Business Models” (gartner.com/en/articles/ai-agents-pc1); Reuters (25 Jun 2025), “Over 40% of agentic AI projects will be scrapped by 2027, Gartner says.”

6 Stanford HAI (13 Apr 2026), AI Index Report 2026.

7 O. Russakovsky et al. (2015), “ImageNet Large Scale Visual Recognition Challenge,” IJCV (human baseline about 5.1%).

8 T. Brown et al. (2020), “Language Models Are Few-Shot Learners,” arXiv:2005.14165.

9 AI Business, “Agentic AI Set to Rise, With New Cybersecurity Risks: Gartner” (A. Litan).

10 Airbus (2016). Breakthrough for future Airbus A320. airbus.com/en/newsroom/news/2016-03-pioneering-bionic-3d-printing

11 Autodesk. General Motors generative design customer story. autodesk.com/customer-stories/general-motors-generative-design

12 ECSS-E-ST-40C, Space engineering: Software. European Cooperation for Space Standardization.

13 Liu, M. et al. (2023). ChipNeMo: Domain-adapted LLMs for chip design. ArXiv:2311.00176.

14 Arora, C. et al. (2025). From domain documents to requirements: retrieval-augmented generation in the space industry. ArXiv:2507.07689.

15 gVisor, application kernel for containers. gvisor.dev

16 Firecracker microVMs. firecracker-microvm.github.io

17 Model Context Protocol. modelcontextprotocol.io

18 LangGraph documentation. langchain-ai.github.io/langgraph

19 Rafailov et al., Direct Preference Optimization, 2023. arXiv:2305.18290

20 Browne et al., A Survey of Monte Carlo Tree Search Methods, 2012. IEEE TCIAIG

21 vLLM documentation. docs.vllm.ai

22 NVIDIA TensorRT-LLM[github.com/NVIDIA/TensorRT-LLM](https://www.google.com/search?q=https%3A%2F%2Fgithub.com%2FNVIDIA%2FTensorRT-LLM)

23 Playwright documentation. playwright.dev

24 Chrome DevTools Protocol. chromedevtools.github.io/devtools-protocol

25 Lewis et al., Retrieval-Augmented Generation, 2020. arXiv:2005.11401

SOURCES

References-2

26 Kwon, W. et al. Efficient Memory Management for Large Language Model Serving with PagedAttention. SOSP 2023. arXiv:2309.06180

27 Dao, T. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. arXiv:2307.08691 (2023)

28 OWASP GenAI Security Project. LLM10:2025 Unbounded Consumption, in OWASP Top 10 for LLM Applications 2025. genai.owasp.org

29 Hugging Face, EleutherAI and Stability AI. Audit shows that safetensors is safe and ready to become the default (Trail of Bits audit). huggingface.co/blog

30 Hugging Face Hub documentation. Pickle Scanning. huggingface.co/docs/hub

31 NVIDIA. NVIDIA Container Toolkit documentation. [docs.nvidia.com/datacenter/cloud-native](https://www.google.com/searchq=https%3A%2F%2Fdocs.nvidia.com%2Fdatacenter%2Fcloud-native)

32 Appsecco. Top 10 Docker Hardening Best Practices (non-root, read-only filesystem, capability dropping). [appsecco.com/blog](https://www.google.com/search?q=https%3A%2F%2Fappsecco.com%2Fblog)

33 Centron. Install Ollama on Ubuntu 24.04 (default bind 127.0.0.1:11434, no built-in authentication). centron.de/tutorials

34 Hugging Face model card, CodeLlama 7B GGUF (GGUF introduced by the llama.cpp team, August 2023). [github.com/ggml-org/llama.cpp](https://www.google.com/search?q=https%3A%2F%2Fgithub.com%2Fggml-org%2Fllama.cpp)

35 Hugging Face. Text Generation Inference: Model safety. huggingface.co/docs/text-generation-inference

THANK YOU

Questions?

Local LLM deployment, air-gapped operations and inference security